Assignment #5 Convolutional Autoencoder for Image Denoising and Deblurring

測量115 F64111134 施淳譯

1. Study the example code, and provide a report describing the methods used in the code and the results.
    a. def preprocess(array): This function converts the gray-scale pixel values into float32, normalizes them to a range of 0–1, and reshapes the array into a tensor of shape (len(array), 28, 28, 1), where the four dimensions represent the number of samples, width, height, and channel of the MNIST images.
    b. def noise(array): This function generates Gaussian random noise with mean 0 and variance 1, with the same shape as the input array. The noise is multiplied by a noise factor and added to the clean images to create noisy versions. The output is clipped to remain within the valid range of 0–1.
    c. def display(array1, array2): This function randomly selects 10 images from array1 and array2 (usually original vs. reconstructed images) and visualizes them in two rows for comparison.
    d. Autoencoder architecture:
        - The input shape is (28, 28, 1).
        - The encoder layer contain two convolution+maxpooling blocks.
        - Convolution layer use 32 filters with 3*3 size, ReLU activation function and same padding, which make sure the image width and height will have the same size as input and the third dimension will depend on the filters we used.
        - Max pooling layers will use 2*2 kernel, and use the same padding to maintain divisibility and ensure the output size remains consistent for decoding.
        - The decoder layer contain two de-convolution layers and one convolution layer.
        - The two de-convolution layers use 32 filters, 3*3 kernel size, ReLU activation, same padding and stride = 2. Acting transpose convolution on the input to reconstruct the corresponding layer in encoder. 3*3 kernel size and 32 filters is the same as the convolution layer in the encoder layer. And the stride = 2 expands input width and height to double of original, which depend on the maxpooling layer.
        - The last convolution layer use 1 filter, 3*3 kernel size, sigmoid activation function and same padding. It will output the (28, 28, 1) image. And use sigmoid activation to change the input to only 0, 1. We will output a binary image.
        - Use Adam optimizer and binary crossentropy to train the model
        - The total parameters we use is 28,353, every parameter is 4 bytes.
    e. Train the autoencoder with original train and test data in Mnist dataset. Use batch size = 128, 50 epochs, shuffle training data and validate use test_data. The latest losses and results are shown below.

```
loss: 0.0627 - val_loss: 0.0625
```

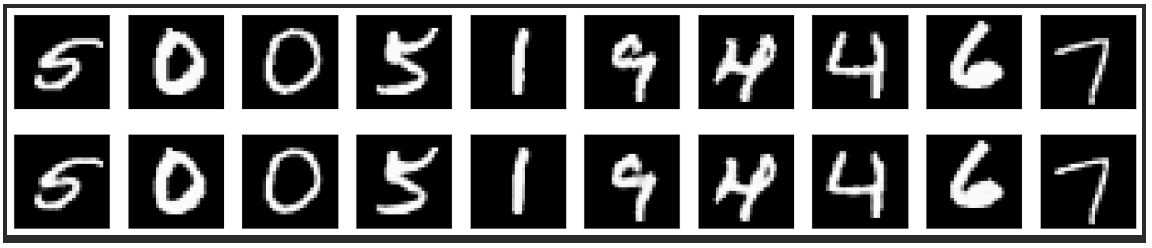


Figure 1,2. Training and Validation Loss and Result in original Mnist Dataset

f. From above, it seems we trained the autoencoder well. And from the result, concern about the edge between white and black pixels. It seems the images processed by autoencoder have more clear edges, and the whole image seems to be more clear. That is, some of the image noise has been removed by autoencoder.

g. Then train the autoencoder with data after noise.The input is noise training data and we want it to approach original images which are y. And we train with epochs = 100, batch size = 128, random shuffle training data and validate input noise test data with original test data . The latest loss and result are shown below.

```
loss: 0.0843 - val_loss: 0.0840
```

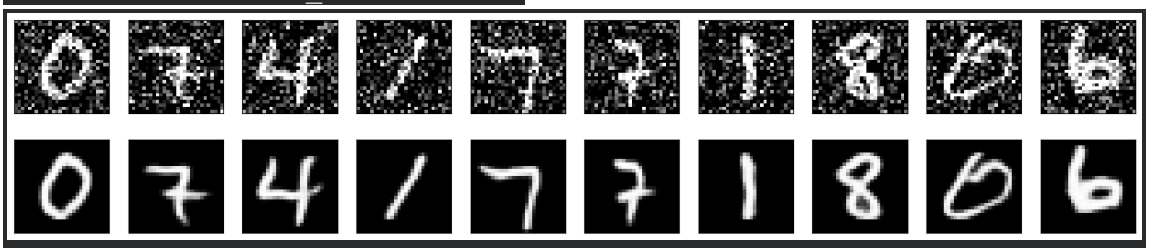


Figure 3, 4. Training and Validation Loss and Result in Noised Mnist Dataset

h. From above, it seems we also trained the autoencoder well, with the small loss in train and validation. And from the result, we can see that the images processed by autoencoder have removed the noise that appears from original images, which is because we train and measure the loss by input noise data to minimize the loss between original non-noised data. The result shows another obvious confidence that we train the autoencoder well.

2. Based on the example code, implement image de-blurring using deep convolutional autoencoder.

a. Image blurring:

```
def blur(array, ksize=7):
    blurred = np.zeros_like(array)
    for i in range(array.shape[0]):
        img = array[i]
        img = img.squeeze()
        bl = cv2.GaussianBlur(img, (ksize, ksize), 0)
        blurred[i] = bl[:, :, np.newaxis]
    return blurred
```

- First create an array with all zeros, and the same shape as array.
- Proceed at every image in the array.

- Since cv2.GaussianBlur only adapts 2D images, and our input image is 3D. Using squeeze to drop the last dimension.
- Use cv2.GaussianBlur to blur image with ksize, which ksize must be odd
- Add the last dimension back to the image.
- The blur image is shown below

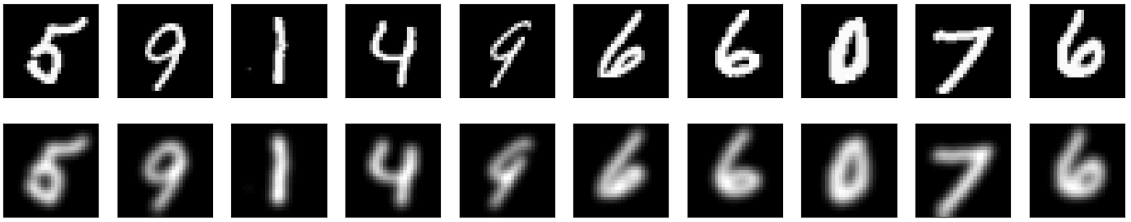


Figure 5. Mnist Images After Blurred

b. Autoencoder: The same autoencoder architecture, optimizer, loss function, and hyperparameters used in the denoising experiment are reused here. The only difference is the dataset: the autoencoder is trained using blurred images as inputs, and the original clean images as targets. Validation is also performed using blurred test images with corresponding clean test images.

```
autoencoder.compile(optimizer="adam",
loss="binary_crossentropy")
autoencoder.fit(
    x=blur_train_data,
    y=train_data,
    epochs=100,
    batch_size=128,
    shuffle=True,
    validation_data=(blur_test_data, test_data),
)
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 32)	9,248
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 32)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 14, 14, 32)	9,248
conv2d_transpose_3 (Conv2DTranspose)	(None, 28, 28, 32)	9,248
conv2d_5 (Conv2D)	(None, 28, 28, 1)	289

Total params: 28,353 (110.75 KB)
Trainable params: 28,353 (110.75 KB)
Non-trainable params: 0 (0.00 B)

Figure 6. Autoencoder Structure

3. Report the image de-blurring processes and results.
   The latest training loss and reconstruction results are shown below:
   loss: 0.0640 - val_loss: 0.0636

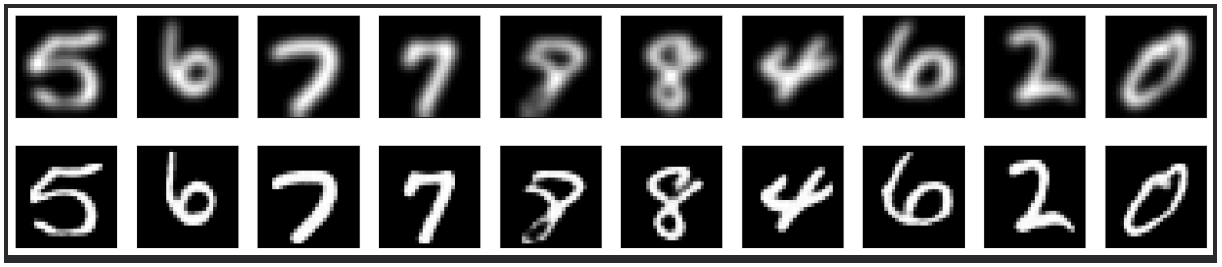


Figure7. Training and Validation loss and Result in Blurred Mnist Dataset

From the training and validation loss above, we can see that our autoencoder seems to be well trained with similar training and validation loss. The reconstructed images also show that the autoencoder performs effectively in de-blurring the input images, successfully restoring clearer digit shapes and sharper edges compared with the blurred inputs.